

Formatting SQL queries with dynamic input.

06 June 2026 10:35

```
String sqlQuery = "insert into employees values(100, 'Dugra', 1000, 'Hyderabad');
```

Or

```
Employee(eno, ename, esal, eaddr)
```

↑ ↑ ↑ ↑ variable in Java

```
String sqlQuery = "insert into employee values("+eno+", '"+ename+"', " + esal + ", '" + eaddr+"'");
```

Or

```
String sqlQuery = String.format("insert into employee values(%d, '%s', %f, '%s')", enno, ename, esal, eaddr);
```

↑ ↑ ↑ ↑ place holders of the variable.

```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
public class InsertRows {
    public static void main(String[] args) {
        String dbUrl = "jdbc:oracle:thin:@//localhost:1521/FREE";
        String username = "sys as sysdba";
        String password = "Calcutta1983";
        Scanner sc = new Scanner(System.in);
        int count = 0;
        try {
            while (true) {
                System.out.println("Enter employee id: ");
                String eid = sc.next();
                sc.nextLine();
                System.out.println("Enter name: ");
                String name = sc.nextLine();
                System.out.println("Enter Department: ");
                String dept = sc.next();
                System.out.println("Enter basic salary: ");
                float bs = sc.nextFloat();
                Connection con = DriverManager.getConnection(dbUrl, username, password);
                Statement stmt = con.createStatement();
                String query = String.format(
                    "INSERT INTO EMPLOYEE VALUES('%s', '%s', '%s', %f)",
                    eid, name, dept, bs);
                int x = stmt.executeUpdate(query);
                count += x;
                System.out.println("\n" + x + " row added.");
                System.out.println("Do you want to insert one more record[yes/no]: ");
                sc.nextLine();
                String option = sc.next();
                if (option.equalsIgnoreCase("No")) {
                    System.out.println("\nNumber of rows stored in the table: " + count);
                    con.close();
                    sc.close();
                    break;
                }
            }
        }
    }
}
```

```

    }
    } catch (SQLException e) {
        e.printStackTrace();
        System.err.println("Connection not established");
    }
}
}

```

Home work:

WAP in JDBC to delete multiple rows from employee table.

Non-select commands

⇒ Create table, drop table, alter table (DDL)

insert $\left\{ \begin{array}{l} \rightarrow \text{Row} \\ \rightarrow \text{Multiple rows} \end{array} \right\}$ (DML)

update $\left\{ \begin{array}{l} \rightarrow \text{single rows} \\ \rightarrow \text{Multiple rows} \end{array} \right\}$

"Select" most commonly used operation.

→ `Class.forName (driver)`

Connection con =

Statement st = Query: "select * from employee where esal > 10000")

```
ResultSet rs = st.executeQuery(query);
```

```
while(rs.next()) {
```

```
System.out.print (... get data from rs);
```

3

Selecting a Particular Columns from a table

eg1: select * from employee;

DB: (eno¹, ename², esal³, eaddr⁴)

RS: (eno, ename, esal, esaldr)

```
while (ms.next()) {
```

rs. (eno, ename, esal, eaddr)

```
while ( rs.next() ) {  
    System.out.println ( rs.getInt(1) + " --- " + rs.getString(2) +  
        " --- " + rs.getDouble(3) + " --- " + rs.getString(4) );  
}
```

eg2: Select ename, eno, eaddr, esal from employees;

DB: (eno¹, ename², esal³, eaddr⁴)

rs: (ename, eno, eaddr, esal)

Resultset column order is based on sql query.

```
while ( rs.next() ) {  
    System.out.println ( rs.getString(1) + " --- " + rs.getInt (2) +  
        " --- " + rs.getString(3) + " --- " + rs.getDouble(4) );  
}
```

eg3: Select ename, eaddr from employees;

DB: (eno¹, ename², esal³, eaddr⁴)

rs: (ename, eaddr)

Resultset column order is based on sql query.

```
while ( rs.next() ) {  
    System.out.println ( rs.getString(1) + " --- " + rs. getString(2) );  
}
```

Homework: W JDBC Programs to test the above query.

Aggregate Functions.

(summarized)

1. To Know the no. of rows: count (*)

eg. select count(*) from employees ;

2. What is the highest salary:

e.g. `select max(sal) from employee;`

3. What is the lowest salary:

e.g. `select min(sal) from employee;`

DB : (eno, ename, sal, addr).

ResultSet rs = st.executeQuery("select min(sal) from employees");

if (rs.next()) {

System.out.println("The minimum salary: " + rs.getDouble());
}

Home work: Write JDBC program to do above tasks.

Select Nth Highest Salaries Employee Information

E001	Pratik Jain	D001	56000	E006	Gargi	D002	70000	1
E002	Naveen Singh	D001	67800					
E003	Sangita Mishra	D004	45700					
E004	Ajay Dewan	D002	34000					
E005	Bali Agarwal	D002	35600					
E006	Gargi	D002	70000					
E007	Garvit	D004	45000					
E008	Yashika	D003	46500					
E009	Amar	D003	43800					

